EDUHUB / ENGINEERING DOCUMENTATION

SOFTWARE ARCHITECTURE

EduHub | Online Learning Platform

01 PRESENTATION	**ASP.NET Core MVC • Razor / Bootstrap 5**
02 BUSINESS LOGIC	**Courses • Learning • Commerce • Certificates**
03 DATA ACCESS	**EF Core • Repositories • Unit of Work**

Architecture style: modular monolith with three logical application layers

Document purpose: the Software Architecture deliverable only.

Prepared from the supplied five-page project specification, with technical recommendations explicitly identified.

SYSTEM ANALYSIS & DESIGN • 2026

SECTION 01

Architecture at a glance

A concise statement of the architectural direction and its scope.

Architecture recommendation

Implement EduHub as an ASP.NET Core MVC modular monolith organized into the three required logical layers: EduHub.Web, EduHub.Services, and EduHub.Data. Deploy the initial web application as one App Service unit, with managed backing services for database, video storage and payments. [S1, R1, R2]

Architecture decision snapshot

Dimension	Proposed choice	Reason
Application style	Three-layer N-Tier modular monolith	Matches the course brief; simpler delivery than microservices.
UI model	ASP.NET Core MVC + Razor + Bootstrap 5	Preserves the explicitly required server-rendered MVC stack.
Business boundaries	Feature modules behind service interfaces	Separates courses, learning, commerce and governance.
Persistence	EF Core + repository interfaces + Unit of Work	Matches patterns specified; supports coordinated writes.
External systems	Stripe, Azure Blob, Azure CDN	Isolates payment and media responsibilities.
Identity	ASP.NET Core Identity + role/policy checks	Enforces Student, Instructor and Admin authorization.

In scope of this architecture document

Architecture drivers; system context; tier responsibilities; modular decomposition; source-code dependencies; design-pattern placement; cross-system interactions; security and operational concerns; trade-offs and open decisions.

Document boundary

Detailed UML component/deployment/sequence diagrams, ERDs, DFDs, API endpoint specifications, wireframes and testing plans are intentionally not delivered here, as they are separate items in the team's System Analysis & Design checklist.

SECTION 02

Drivers, constraints & assumptions

What determines this architecture, and which choices are not yet confirmed by the brief.

Architecture driver	Design response	Source status
Three user roles: Student, Instructor, Admin	Identity roles plus resource ownership and policy checks.	Required [S1]
Create, approve, publish and purchase courses	Explicit service-level workflows and state checks.	Required / inferred [S1]
Video upload, processing and protected viewing	Private blob media; short-lived authorized access tokens.	Required + recommendation [S1, R4]
Multiple paid courses, coupons and receipts	Commerce module, pricing strategies and verified payment events.	Required + recommendation [S1, R5]
Progress, quizzes and certificates	Persistent learning state; completion-eligibility rules.	Required [S1]
Delivery within a four-week learning plan	Monolith first; defer complex distributed processing.	Pragmatic recommendation

Assumptions register

ID	Assumption / decision awaiting team confirmation
A-01	Azure SQL Database is the recommended relational store; the source specifies EF Core, not a database product.
A-02	The first release is a single deployable web app. A separate video-processing worker is optional.
A-03	An instructor submits a draft for admin approval before it becomes publicly discoverable; exact workflow rules are not prescribed.
A-04	A successful, verified Stripe server-side event controls paid enrollment; no client redirect is authoritative.
A-05	Video processing, email transport, caching product and background scheduler remain technology selections.

SECTION 03

System context

System boundary, primary actors and external dependencies.

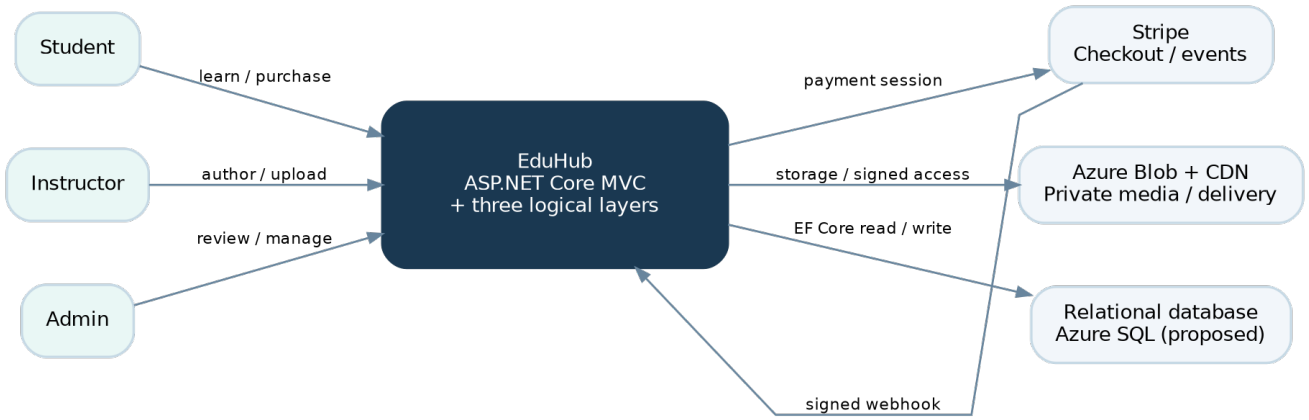


Figure 1. Logical context diagram. The relational database product is proposed; Blob, CDN and Stripe are specified in [S1].

Participant / system	Architectural interaction
Student	Browse/enroll, pay, stream lessons, submit quizzes, see progress, obtain certificates.
Instructor	Create structured courses, upload media, manage pricing and student engagement.
Admin	Moderate courses, manage users/categories and view platform analytics.
Stripe	Receives checkout requests and sends independently verified payment events.
Blob / CDN	Stores original/processed content and serves protected media through controlled links.
Relational store	Persists identities, catalog, commerce, learning state and certificate metadata.

Trust boundary

Browsers and Stripe callbacks are untrusted input sources. Authorization and business decisions occur server-side; direct database access from clients is prohibited.

SECTION 04

Three-layer logical architecture

The MVC pattern exists inside the presentation layer; it is not a substitute for N-Tier architecture. [R1, R2]

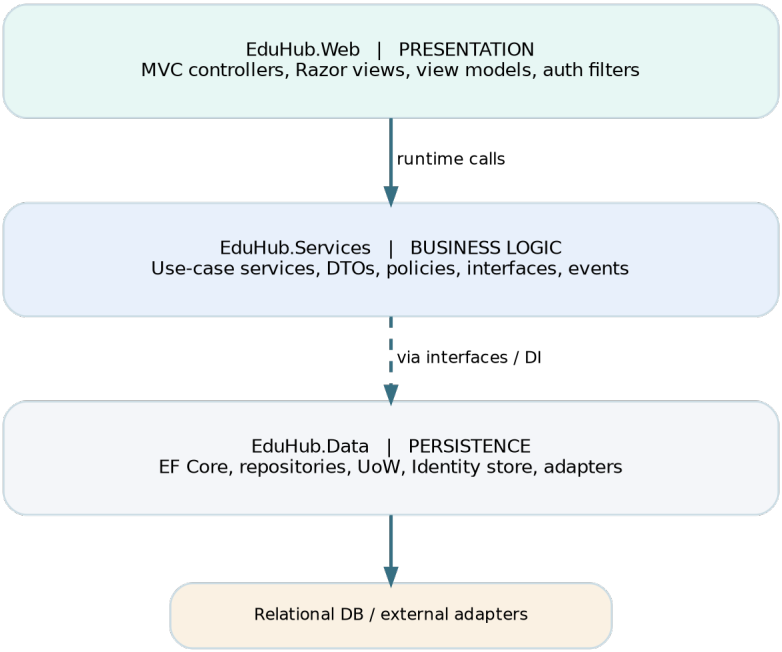


Figure 2. Runtime responsibilities. The dashed edge denotes a logical service-to-repository call resolved through dependency injection.

Project / layer	Owns	Must not own
EduHub.Web Presentation	Routing, controllers, Razor pages, client-side JS (Video.js / Chart.js / Progress.js), request validation, security middleware, composition root.	SQL queries; pricing rules; certificate eligibility; direct Stripe fulfillment.
EduHub.Services Business logic	Course moderation, enrollment, checkout orchestration, pricing policies, progress, assessments, certificates; domain contracts, DTOs and events.	HttpContext dependence; EF Core provider logic; Razor rendering.
EduHub.Data Data access	EF Core DbContext, schema mappings, repository and UoW implementations, Identity persistence, query optimization; infrastructure implementations.	Controllers or UI logic; business policy selection.

Clarification: layer vs tier

These are three logical application layers/projects. They can run in the same ASP.NET Core process. SQL, Stripe and Azure Storage are separate infrastructure/services; separate network deployments of each application layer are not required. [R1]

SECTION 05

Source dependencies & solution layout

Avoid compile-time circular dependencies while honoring the three-project requirement.

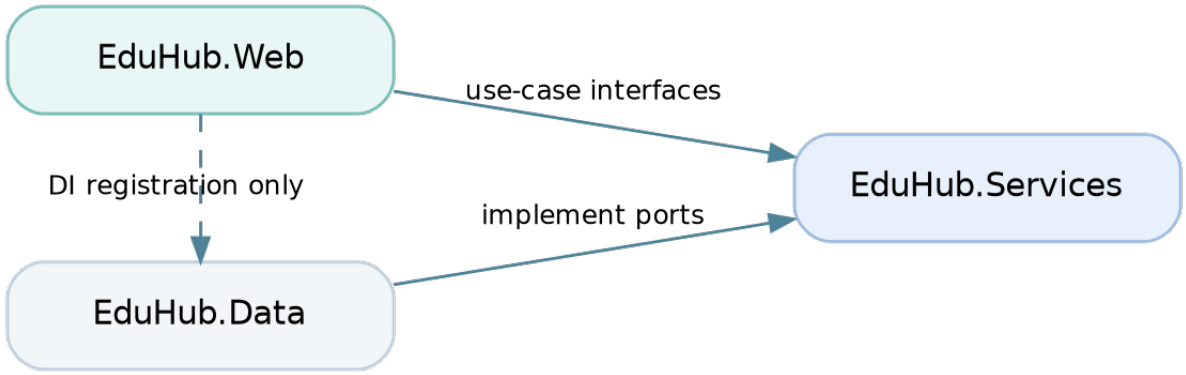


Figure 3. Compile-time project references: Web → Services; Data → Services; Web → Data only for infrastructure registration.

Code location	Recommended contents
EduHub.Web/	Areas/Student, Areas/Instructor, Areas/Admin; Controllers; Views; ViewModels; wwwroot; middleware; Program.cs.
EduHub.Services/	Modules/{Catalog,Learning,Commerce,Assessment,Certificates,Governance}; interfaces; business models; DTOs; policies; events.
EduHub.Data/	Persistence/EduHubDbContext; Configurations; Repositories; UnitOfWork; Identity; Integrations; Migrations.

Dependency rules

- Controllers depend on service interfaces; never inject repositories or DbContext into controllers.
- Services depend on persistence/provider ports defined in Services; Data implements those ports.
- The Web composition root configures DI for services, persistence and external adapters; no Data API usage in controllers.
- Keep transport view models in Web, use-case DTOs and domain behavior in Services, and EF configurations in Data.

> **Implementation caveat**
>
> Identity persistence may use an ApplicationUser mapped inside Data. Expose user identifiers and identity operations through service interfaces/DTOs rather than leaking Identity or EF models across boundaries.

SECTION 06

Business modules & ownership

Feature boundaries are internal modules, not separately deployed microservices.

Module	Service responsibility	Key collaborators
Identity & Access	Registration, sign-in, roles, user profiles and authorization policy checks.	ASP.NET Core Identity / Data
Catalog & Authoring	Course metadata, categories, sections, lessons, drafts, publish-ready validation, ratings/Q&A.;	Media; Governance
Governance	Course approval decisions, user/category moderation and platform settings.	Catalog; Identity
Commerce	Cart, pricing/coupons, checkout, orders, receipts, payment-event reconciliation.	Stripe; Enrollment
Enrollment & Learning	Free enrollment, paid entitlement, lesson completion, notes, bookmarks and course progress.	Commerce; Assessment
Assessment	Quiz definition, attempts, grading, passing rules and results.	Learning
Certificates	Eligibility check, unique verification token, PDF generation and verification page.	Learning; Assessment
Media	Secure uploads, asset metadata, processing status, playback authorization and signed delivery.	Blob / CDN
Analytics & Notifications	Read-oriented reporting; progress events and student/instructor alerts.	All relevant modules

Data ownership policy

Each module owns its invariants and orchestrates changes through its service surface. Repositories may share a single EF Core DbContext and database, but cross-module writes must pass through the relevant service or an explicit orchestrator. Reporting may use read-only projections without bypassing authorization.

> **Practical design rule**
>
> One deployable, many cohesive modules. No distributed service mesh, separate service databases, or event broker is necessary for the initial graduation scope.

SECTION 07

Key interactions

Representative end-to-end paths that validate the architectural boundaries.

A. Paid course purchase

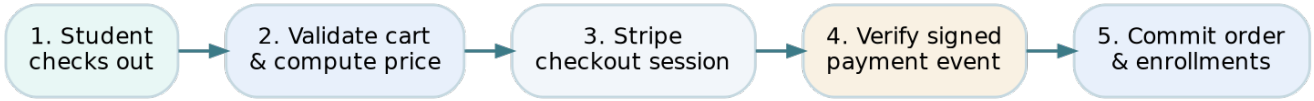


Figure 4. Payment completion flow. The checkout redirect is for UX; only authenticated server-side verification grants paid access. [R5]

Boundary	Required architectural behavior
Web → Services	Student submits cart; Services rechecks ownership, published status, prices and discounts.
Services → Stripe	Provider adapter creates a checkout session tied to the internal pending order.
Stripe → Web	Webhook endpoint checks raw payload signature, event type and event uniqueness.
Services → Data	Validated successful payment atomically records state and enrollments; retries are idempotent.

B. Authorized learning & progress

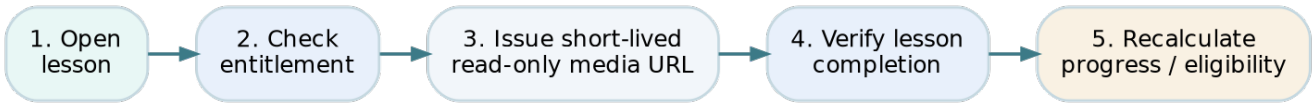


Figure 5. Learning flow. UI video-play signals alone must not be treated as trusted completion evidence.

Progress rules should be deterministic: count eligible completed lessons; account for required quiz scores; emit a progress-updated event. The certificate module decides whether completion meets certificate criteria.

SECTION 08

Media & certificate processing

Large assets and generated documents require different persistence and trust rules.

Video pipeline: proposed logical design

Step	Responsible component	Control
1. Request upload	Media service	Authorize instructor ownership; validate file type/size; create asset record.
2. Transfer	Browser → private Blob container	Short-lived constrained upload credential; no storage key in browser.
3. Process	Background processing adapter (technology TBD)	Compress/transcode and generate thumbnails; update processing status.
4. Publish	Catalog + Media services	Prevent publication/playback until required media assets are ready.
5. Playback	Learning service → media adapter	Verify enrollment; issue read-only time-limited media URL; CDN as appropriate.

Certificate pipeline

Trigger	Business rule	Output
Learning completion event	Student must meet configured lesson and quiz requirements.	Eligible completion record
Certificate issuance	Avoid duplicate issuance; generate unique hard-to-guess verification code; render PDF with the specified iTextSharp library (verify license/version compatibility).	Certificate metadata + PDF
Public verification request	Return only permitted public fields; detect invalid/revoked identifiers.	Verification result page

Important integration constraints

Azure Blob Storage is a storage service, not a video transcoder; the processing tool must be separately selected. Prefer user-delegation SAS when feasible, use HTTPS, grant minimum permissions, and keep Blob containers private. [R4]

Failure isolation

Video processing and PDF generation should be retryable operations with explicit states (Pending, Processing, Ready, Failed). If processing is asynchronous, the application must expose status and retry policy; avoid long-running video conversions inside a normal HTTP request.

SECTION 09

Design-pattern implementation map

The required patterns should solve identifiable problems rather than add boilerplate.

Pattern	Placement	Concrete application & guardrail
N-Tier architecture	Across Web / Services / Data	Isolate UI, application policy and persistence; explicit dependency boundaries.
Repository	Services interfaces; Data implementations	Use capability-specific contracts (e.g. ICourseRepository) for queries and aggregates; avoid unrestricted generic CRUD.
Unit of Work	Services port; Data implementation	Commit related order/enrollment updates in one database transaction; EF Core DbContext already exhibits UoW semantics. [R3]
Strategy	Services/Pricing	IPricingStrategy / discount rules for coupon, promotion and regular price; validate combinations and precedence.
Observer	Services domain events	LessonCompleted → progress update / notification. In-process handlers for MVP; durable queue/outbox only if reliability needs justify it.
Dependency Injection	Web composition root	Construct services using typed interfaces; register adapters; support mocking/test seams.

Suggested contracts (illustrative, not an API specification)

Interface	Representative operations
ICourseService	CreateDraftAsync, SubmitForApprovalAsync, PublishAsync
ICheckoutService	CreateCheckoutAsync, ConfirmPaymentEventAsync
IProgressService	RecordLessonCompletionAsync, GetCourseProgressAsync
IMediaStorage	CreateUploadGrantAsync, CreatePlaybackGrantAsync
IUnitOfWork	SaveChangesAsync (or CommitAsync with clear semantics)

Avoid a repository anti-pattern

Do not create an additional database transaction for every read. EF Core SaveChanges is transactional for a single call when the provider supports transactions; explicit transactions belong around multi-operation invariants. [R3]

SECTION 10

Security, reliability & operability

Architecture-wide policies spanning controllers, services and persistence.

Concern	Baseline architectural control
Authentication & roles	ASP.NET Core Identity with Student / Instructor / Admin; secure cookies; HTTPS; account lockout and password rules. [R6]
Authorization	Policies + object ownership; enrollment entitlement gates media, notes, quizzes and certificate access.
Input security	Server-side validation, MVC antiforgery for unsafe form actions, encoded Razor output, restrictive upload validation.
Payment integrity	Stripe signed webhook validation using raw request, deduplication, server-side amount reconciliation. [R5]
Media confidentiality	Private blobs, scoped short-expiry user-delegation SAS where supported, no long-lived public URLs. [R4]
Data correctness	Database uniqueness constraints for enrollment/payment events/certificates; concurrency tokens where needed.
Observability	Structured logs, correlation IDs, payment/event audit trail, processing state and error metrics; avoid PII in logs.
Resilience	Retries with backoff for transient provider errors, idempotent handlers, appropriate timeouts and graceful degradation.
Secret management	Environment/App Service managed settings or Key Vault (recommended); do not commit provider keys.

Proposed quality attributes for later validation

Attribute	Design intent (target to be defined by team)
Maintainability	One module owns each policy; no controllers reaching DbContext or calling Stripe directly.
Scalability	Stateless web tier where possible; CDN media delivery; optional worker extraction later.
Availability	Payment events must be replayable; incomplete uploads and generation tasks recoverable.
Performance	Paginate course lists; index common reads; cache public catalog data if profiling supports it.

SECTION 11

Logical hosting & evolution

A hosting view to explain architecture boundaries; not a detailed deployment diagram.

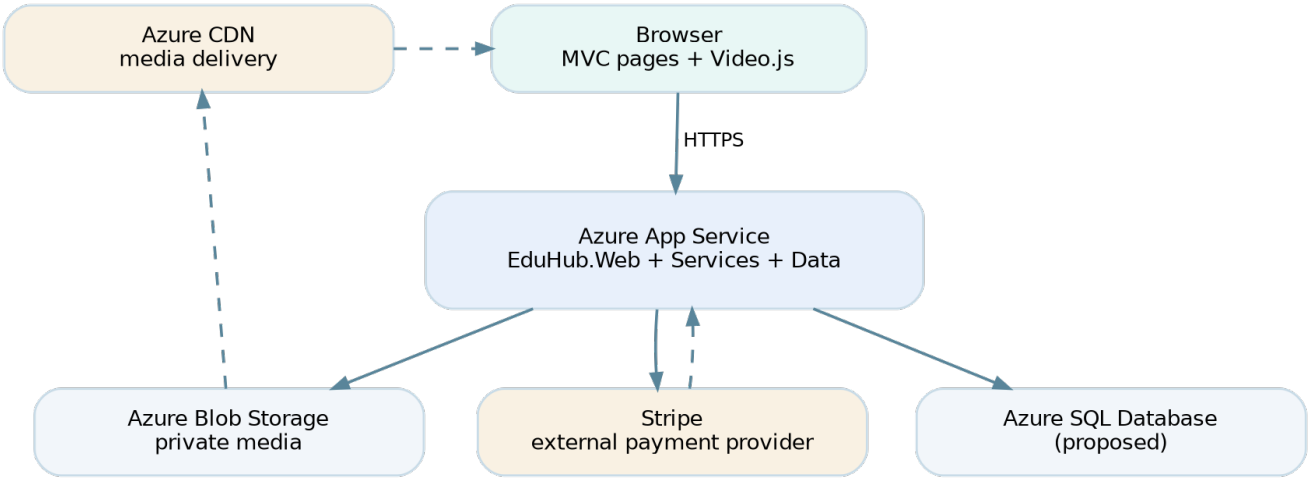


Figure 6. Proposed hosting view. Azure App Service and CDN/Blob are named in [S1]; Azure SQL is a recommended choice. [R7]

Decision	Why it fits now	When to reconsider
Modular monolith, not microservices	One team, shared transactions and a short project schedule.	Independent teams, scaling and deployability requirements emerge.
Server-rendered MVC, not SPA-first	Exactly matches required ASP.NET Core MVC stack.	Rich standalone clients or multiple consumers become mandatory.
Single relational store	Transactional payment, course, identity and enrollment data.	Evidence of scaling or isolation bottlenecks.
Background processing optional	HTTP handlers stay lightweight; processing can start with a modest job system.	Videos are large, high-volume or require strict delivery SLAs.

SECTION 12

Architecture review checklist

Criteria the team can use to approve this architecture before detailed design begins.

Check	Acceptance condition
☐ Layer integrity	Web controllers use Services; Services use abstractions; Data implements persistence ports.
☐ Core functionality	All required EduHub features map to one owning module and authorized workflow.
☐ Secure commerce	Paid access cannot be granted from browser redirects; replayed Stripe events are safe.
☐ Secure media	Media containers are private; upload and playback grants are short-lived and scoped.
☐ Identity integrity	Student, Instructor and Admin role boundaries and owner checks are enforceable.
☐ Reliable completion	Lesson and quiz outcomes deterministically control progress and certificates.
☐ Operability	Errors, retries and provider outages have observable states and recovery strategies.
☐ Diagram consistency	Code dependencies, logical data access and hosting views describe the same solution.

Open architectural decisions to close

Before implementation: confirm relational database choice; define catalog approval lifecycle; identify the video transcoding/thumbnail component; decide synchronous vs durable notifications and certificate issuance; establish pricing/coupon precedence; define cache and secret-storage strategy.

References

[S1] EduHub_ProjectDetails(1).pdf, supplied project specification, pp. 1–5. (Primary project requirements.)

[R1] Microsoft Azure Architecture Center — N-tier architecture style.

[R2] Microsoft Learn — Overview of ASP.NET Core MVC.

[R3] Microsoft Learn — EF Core transactions; DbContext behavior.

[R4] Microsoft Learn — Grant limited access to data with SAS.

[R5] Stripe Docs — Receive Stripe events in your webhook endpoint.

[R6] Microsoft Learn — Role-based authorization in ASP.NET Core.

[R7] Microsoft Learn — Azure App Service overview.